

Relatório — problema 1: O Jantar dos Filósofos

1. Solução adotada

O problema do Jantar dos Filósofos foi implementado em Go com $N = 5$ filósofos. Cada filósofo é uma goroutine independente que alterna entre os estados de pensar, ficar com fome e comer. Os garfos são representados por channels com buffer de tamanho 1, simulando mutexes — receber do channel equivale a adquirir o garfo, e enviar de volta equivale a liberá-lo.

Foram implementadas três versões: uma versão base que demonstra a ocorrência de deadlock, e duas estratégias que o eliminam.

2. Versão Base — Com Deadlock

Nesta versão, todos os filósofos seguem a mesma ordem de aquisição: pegam primeiro o garfo à direita e depois tentam pegar o garfo à esquerda. Quando todas as goroutines são escalonadas simultaneamente, cada filósofo consegue adquirir seu garfo direito, mas fica bloqueado esperando o garfo esquerdo — que está retido pelo vizinho, que por sua vez também está esperando. Forma-se assim uma espera circular entre todos os filósofos, satisfazendo as quatro condições de Coffman: exclusão mútua, posse e espera, não-preempção e espera circular. O programa entra em deadlock e fica travado indefinidamente, sem que nenhum filósofo consiga comer.

Starvation não chega a ocorrer de forma independente nesta versão, pois o deadlock impede qualquer progresso antes que isso aconteça.

3. Estratégia 1 — Assimetria de Ordem de Aquisição

Para eliminar o deadlock, adotou-se a estratégia de assimetria na ordem de aquisição dos garfos. Todos os filósofos pegam o garfo direito antes do esquerdo, exceto o filósofo 0, que inverte essa ordem. Essa quebra de simetria elimina a condição de espera circular de Coffman: não é possível que todos os filósofos estejam simultaneamente aguardando o garfo do vizinho, pois o filósofo 0 compete pelo garfo em direção oposta aos demais, quebrando o ciclo.

Starvation não ocorre nesta implementação, pois o `time.Sleep` garante que nenhum filósofo monopoliza os garfos indefinidamente.

Análise de Fairness — Estratégia 1

Os dados coletados em 3 execuções de 100 iterações confirmam fairness perfeita:

Filósofo Execução 1 Execução 2 Execução 3 Média

0	100	100	100	100
1	100	100	100	100
2	100	100	100	100
3	100	100	100	100
4	100	100	100	100

Todos os filósofos comeram exatamente 100 vezes em todas as execuções, demonstrando ausência de starvation e distribuição completamente justa do acesso aos recursos.

4. Estratégia 2 - Uso de monitor

A segunda estratégia implementa um monitor clássico composto por um mutex (o `sync.Mutex`), uma variável de condição (`sync.Cond`) e os garfos, que são representados como canais com buffer 1. A lógica central dessa estratégia está na função `PickFork`, que tenta adquirir os dois garfos de forma atômica usando um `select` aninhado: o filósofo adquire o garfo esquerdo e, em seguida, tenta adquirir o direito. Se o garfo direito já estiver em uso, o esquerdo é devolvido imediatamente e o filósofo chama o `Cond.Wait()`, que libera o mutex e suspende a goroutine. Ao liberar os garfos (função `ReleaseFork`), um `cond.Broadcast()` acorda todos os filósofos que estão dormentes, dando a eles mais uma chance para tentarem pegar o garfo.

A respeito do deadlock, ele não ocorre aqui pois a condição de posse e espera é eliminada: o filósofo nunca segura um garfo enquanto aguarda o outro. Se não conseguir os dois garfos atomicamente, ele devolve o esquerdo antes de chamar `cond.Wait()`. Sem essa condição satisfeita, não há como formar o ciclo de espera circular que caracteriza o deadlock. O mutex garante exclusão mútua no acesso ao estado interno, e o `cond.Wait()` garante que o filósofo só retome quando sinalizado, evitando busy wait.

A respeito do starvation, ele pode ocorrer nessa implementação. O mecanismo `cond.Broadcast()` acorda todas as goroutines suspensas ao mesmo tempo, sem qualquer garantia de ordem ou prioridade. Desse jeito, após o broadcast, todas as goroutines competem pela reaquisição do mutex, e o escalonador do Go não oferece fairness entre goroutines nessa disputa. Além disso,

pela falta do time.sleep nessa solução, na função Dine, um filósofo que acabou de comer pode readquirir o mutex imediatamente e voltar a disputar os garfos antes que os filósofos que estavam suspensos consigam reagir ao Broadcast.

4.1. Análise de Fairness

Para evidenciar a assimetria de acesso, medimos o tempo (em ms) que cada filósofo levou para completar suas 10.000 (melhor para mostrar a discrepância) refeições em 3 execuções distintas:

Filósofo	Execução 1 (ms)	Execução 2 (ms)	Execução 3 (ms)	Media (ms)
0	77.092	77.084	96.648	83.608
1	78.026	80.769	93.837	84.211
2	68.072	69.247	102.532	79.950
3	72.747	52.793	99.773	75.104
4	51.441	80.268	102.394	78.034

5. Comparação entre soluções

A estratégia 1 (Assimetria) foca na justiça (fairness) absoluta. O uso de time.sleep e a quebra de simetria na ordem dos garfos garantem que todos os filósofos comam exatamente a mesma quantidade de vezes, tornando o sistema previsível e equilibrado.

Ja a estratégia 2 (Monitor) prioriza a disputa agressiva por recursos. Por não utilizar pausas obrigatórias e depender da reaquisição imediata do mutex, ela permite que alguns filósofos consigam comer mais rápido que outros dependendo do escalonador, o que pode aumentar o desempenho geral, mas sacrifica a igualdade perfeita entre os participantes.

Relatório — Problema 2: Jogo de Trinca

Programação Concorrente

Relatório — Problema 2: Jogo de Trinca

1. Solução adotada

O jogo foi implementado em Go seguindo o modelo de comunicação por channels. Cada jogador é uma goroutine que roda de forma independente e se comunica exclusivamente trocando mensagens — sem nenhuma variável compartilhada entre eles (exceto o mutex usado só para não embaralhar o log no terminal).

A topologia de comunicação é um anel: cada jogador possui um channel de onde recebe cartas (vindo do vizinho da direita) e um channel para onde envia seu descarte (para o vizinho da esquerda). Além disso, existe um channel global `batidaCh` por onde o jogador que formar trinca avisa um árbitro, que então envia o sinal de encerramento para todos via channels individuais de reagir.

A estratégia de descarte é simples: o jogador descarta sempre a carta menos frequente na mão, priorizando manter pares.

2. Decisões de projeto

Channels de cartas com buffer = 1. Essa foi a decisão mais importante. Channels sem buffer causam deadlock no anel. Com buffer 1, o jogador deposita a carta e segue em frente sem precisar esperar o vizinho estar pronto para receber.

select em todas as operações bloqueantes. Toda vez que um jogador tenta enviar ou receber em um channel, o código usa select com um case <-reagir alternativo.

Árbitro com envios paralelos. O árbitro dispara uma goroutine separada para enviar o sinal de encerramento para cada jogador.

defer wg.Done(). A primeira linha de cada goroutine-jogador registra o Done com defer.

3. Análise de deadlock

Deadlock acontece quando duas ou mais goroutines ficam bloqueadas esperando uma pela outra, sem que nenhuma consiga avançar.

Ponto 1 — Anel circular com channels unbuffered.

Se todos os channels de carta fossem unbuffered, todos os jogadores tentariam enviar ao mesmo tempo e nenhum avançaria para receber — deadlock clássico em anel.

Ponto 2 e 3 — Jogo encerra durante envio ou recebimento.

Se o árbitro sinalizar o fim enquanto um jogador está bloqueado tentando enviar ou receber, sem tratamento ele ficaria preso indefinidamente.

Ponto 4 — Dois jogadores tentam bater ao mesmo tempo.

Se dois jogadores formassem trinca simultaneamente, os dois tentariam enviar para batidaCh.

Ponto 5 — Árbitro travado enviando sinais em série.

A solução foi lançar uma goroutine por jogador para fazer cada envio de forma independente e paralela.

4. Análise de starvation

Starvation é quando uma goroutine nunca consegue avançar porque outras sempre ficam na frente.

Nos channels de carta, não há disputa: cada channel conecta exatamente um produtor a um consumidor.

5. Exemplo de execução

=== JOGO DE TRINCA ===

Jogador 1 começa com: [2 9 2]

Jogador 2 começa com: [10 7 12]

Jogador 3 começa com: [6 8 12]

Jogador 4 começa com: [1 6 1]

Jogador 1 descarta 9

Jogador 2 descarta 7

...

Jogador 2 BATEU com trinca de 3!

Jogador 2 bateu! Aguardando reações...

Jogador 4 reagiu (posição 1)

Jogador 1 reagiu (posição 2)

Jogador 3 reagiu (posição 3)

Jogador 3 foi o ÚLTIMO a reagir — perdeu!

Jogador 2 venceu a rodada!

=== FIM DO JOGO ===